```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('sales_data_sample.csv',encoding='unicode_escape')
df.head()
```

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE	STATUS	QTR_ID	MONTH_ID	YEAR_ID	...	ADDRESSLINE
0	10107	30	95.70	2	2871.00	2/24/2003 0:00	Shipped	1	2	2003	...	897 Lor Airport Avenu
1	10121	34	81.35	5	2765.90	5/7/2003 0:00	Shipped	2	5	2003	...	59 rue d l'Abbay
2	10134	41	94.74	2	3884.34	7/1/2003 0:00	Shipped	3	7	2003	...	27 rue d Colonel Pier Av
3	10145	45	83.26	6	3746.70	8/25/2003 0:00	Shipped	3	8	2003	...	78934 Hillsi D
4	10159	49	100.00	14	5205.27	10/10/2003 0:00	Shipped	4	10	2003	...	7734 Stror S

5 rows × 25 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2823 entries, 0 to 2822
Data columns (total 25 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   ORDERNUMBER       2823 non-null   int64
 1   QUANTITYORDERED   2823 non-null   int64
 2   PRICEEACH         2823 non-null   float64
 3   ORDERLINENUMBER   2823 non-null   int64
 4   SALES             2823 non-null   float64
 5   ORDERDATE         2823 non-null   object
 6   STATUS            2823 non-null   object
 7   QTR_ID            2823 non-null   int64
 8   MONTH_ID          2823 non-null   int64
 9   YEAR_ID           2823 non-null   int64
 10  PRODUCTLINE       2823 non-null   object
 11  MSRP              2823 non-null   int64
 12  PRODUCTCODE       2823 non-null   object
 13  CUSTOMERNAME      2823 non-null   object
 14  PHONE             2823 non-null   object
 15  ADDRESSLINE1      2823 non-null   object
 16  ADDRESSLINE2      302 non-null    object
 17  CITY              2823 non-null   object
 18  STATE             1337 non-null   object
 19  POSTALCODE        2747 non-null   object
 20  COUNTRY           2823 non-null   object
 21  TERRITORY         1749 non-null   object
 22  CONTACTLASTNAME   2823 non-null   object
 23  CONTACTFIRSTNAME  2823 non-null   object
 24  DEALSIZE          2823 non-null   object
dtypes: float64(2), int64(7), object(16)
memory usage: 551.5+ KB
```

```
df_drop  = ['ADDRESSLINE1', 'ADDRESSLINE2', 'POSTALCODE', 'CITY', 'TERRITORY', 'PHONE', 'STATE', 'CONTACTFIRSTNAME', 'CONTACTLASTN
df = df.drop(df_drop, axis=1)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2823 entries, 0 to 2822
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   QUANTITYORDERED  2823 non-null   int64
 1   PRICEEACH        2823 non-null   float64
 2   ORDERLINENUMBER  2823 non-null   int64
 3   SALES            2823 non-null   float64
 4   ORDERDATE        2823 non-null   object
 5   STATUS           2823 non-null   object
 6   QTR_ID           2823 non-null   int64
 7   MONTH_ID         2823 non-null   int64
 8   YEAR_ID          2823 non-null   int64
 9   PRODUCTLINE      2823 non-null   object
 10  MSRP             2823 non-null   int64
 11  PRODUCTCODE      2823 non-null   object
 12  COUNTRY          2823 non-null   object
```

```
 13  DEALSIZE         2823 non-null   object
dtypes: float64(2), int64(6), object(6)
memory usage: 308.9+ KB
```

```
for col in df.columns.values:
    print(df[col].value_counts())
```

```
Motorcycles         331
Planes              306
Trucks and Buses    301
Ships               234
Trains               77
Name: count, dtype: int64
MSRP
118    104
99     103
136     80
62      78
68      77
      ...
73      23
71      22
92      22
170     22
41      22
Name: count, Length: 80, dtype: int64
PRODUCTCODE
S18_3232    52
S12_1666    28
S10_4962    28
S18_1097    28
S10_1949    28
            ..
S18_2248    22
S24_2887    22
S18_4933    22
S18_4409    22
S24_3969    22
Name: count, Length: 109, dtype: int64
COUNTRY
USA            1004
Spain           342
France          314
Australia       185
UK              144
Italy           113
Finland          92
Norway           85
Singapore        79
Canada           70
Denmark          63
Germany          62
Sweden           57
Austria          55
Japan            52
Belgium          33
Switzerland      31
Philippines      26
Ireland          16
Name: count, dtype: int64
DEALSIZE
Medium    1384
Small     1282
Large      157
Name: count, dtype: int64
```

```
df.drop(columns=['ORDERDATE','STATUS','MONTH_ID','QTR_ID','YEAR_ID'],inplace=True)
df.head()
```

	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	PRODUCTLINE	MSRP	PRODUCTCODE	COUNTRY	DEALSIZE
0	30	95.70	2	2871.00	Motorcycles	95	S10_1678	USA	Small
1	34	81.35	5	2765.90	Motorcycles	95	S10_1678	France	Small
2	41	94.74	2	3884.34	Motorcycles	95	S10_1678	France	Medium
3	45	83.26	6	3746.70	Motorcycles	95	S10_1678	USA	Medium
4	49	100.00	14	5205.27	Motorcycles	95	S10_1678	USA	Medium

Next steps: Generate code with df | New interactive sheet

```
from sklearn.preprocessing import LabelEncoder
def convert_categories(col):
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].values)
```

```
categories = ['PRODUCTLINE','PRODUCTCODE','COUNTRY','DEALSIZE']
for col in categories:
    convert_categories(col)
```

```
df.head()
```

	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	PRODUCTLINE	MSRP	PRODUCTCODE	COUNTRY	DEALSIZE
0	30	95.70	2	2871.00	1	95	0	18	2
1	34	81.35	5	2765.90	1	95	0	6	2
2	41	94.74	2	3884.34	1	95	0	6	1
3	45	83.26	6	3746.70	1	95	0	18	1
4	49	100.00	14	5205.27	1	95	0	18	1

Next steps: Generate code with df | New interactive sheet

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
data = sc.fit_transform(df)
```

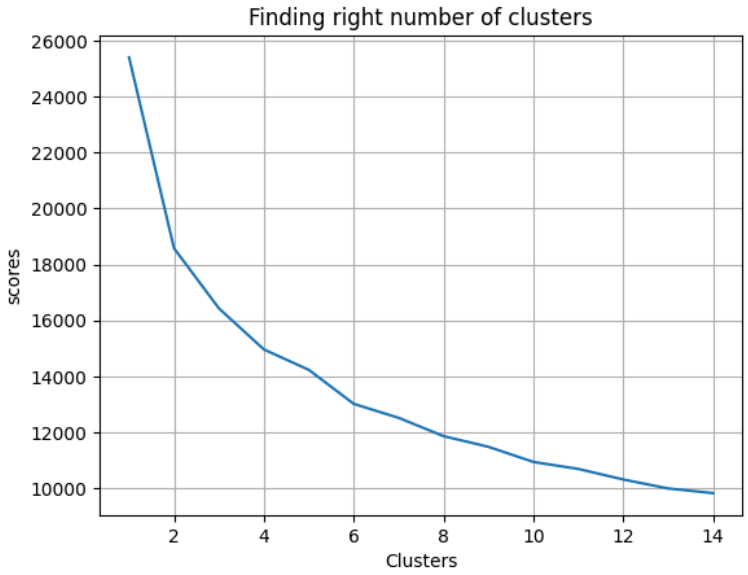
Elbow Method

Finding optimal numbers of clusters is elbow method

For each value of K, we are calculating WCSS (Within-Cluster Sum of Square). WCSS is the sum of squared distance between each point and the centroid in a cluster.When we plot the WCSS with the K value, the plot looks like an Elbow

```
from sklearn.cluster import KMeans
wcss = []
for k in range(1,15):
    kmeans = KMeans(n_clusters=k,init='k-means++',random_state=15)
    kmeans.fit(data)
    wcss.append(kmeans.inertia_)
```

```
k = list(range(1,15))
plt.plot(k,wcss)
plt.xlabel('Clusters')
plt.ylabel('scores')
plt.title('Finding right number of clusters')
plt.grid()
plt.show()
```



At k=4, the graph starts to move almost parallel to the X-axis. The K value corresponding to this point is the optimal K value or an optimal number of clusters.

```
Start coding or generate with AI.
```